Lightweight DSP in ordinary C

Q15 dot product and FIR, using packed multiply-accumulate intrinsics.

```
int64_t sum = 0;
size_t i = 0;
for (; count - i >= 2; i += 2)
  {
    int16x2_t av = __riscv_pld_i16x2 ((int16_t *) (a + i));
    int16x2_t bv = __riscv_pld_i16x2 ((int16_t *) (b + i));
    sum = __riscv_pm2wadda_i64 (sum, av, bv);
  }
```

LOAD

Two signed Q15 samples from each input.

MULTIPLY–ACCUMULATE

Add both products to a 64-bit running sum.

REUSE FOR FIR

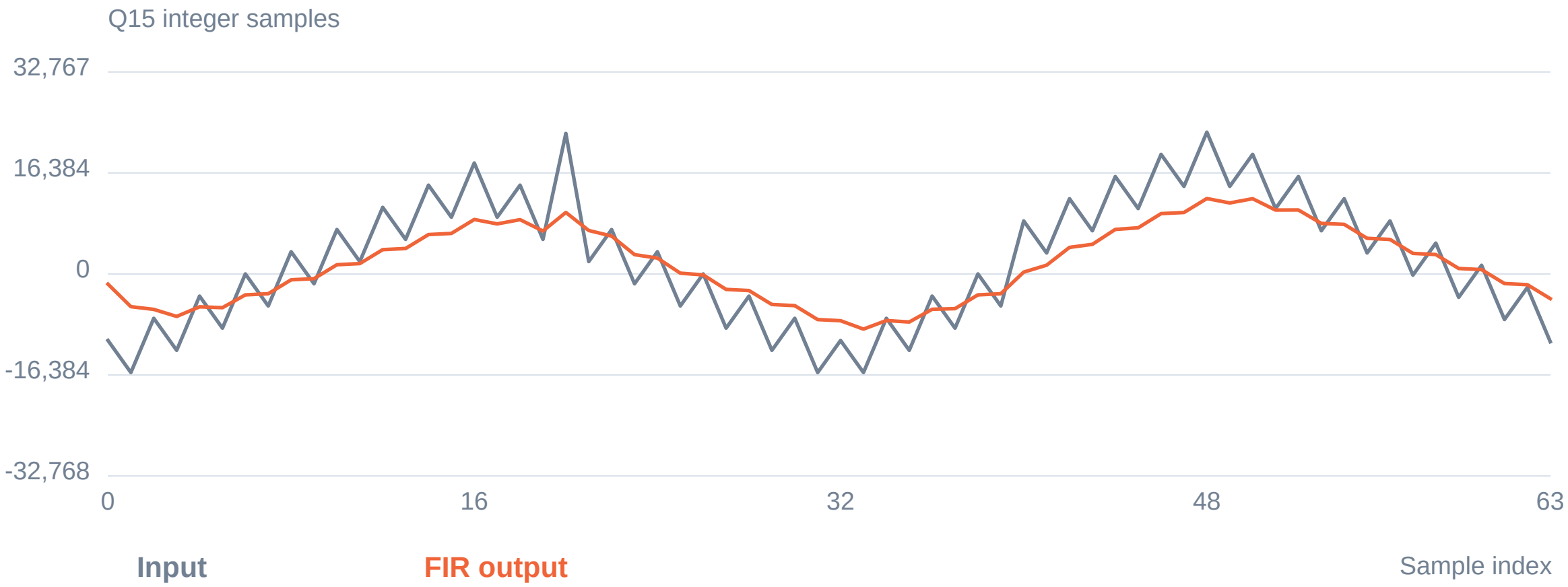
Apply the dot product to each sample window.

Two samples per packed MAC on RV32; four on RV64.

The same intrinsic loop supports a dot product and each FIR sample window.
Complete C kernels and test inputs are included.

Filter a signal with a short FIR

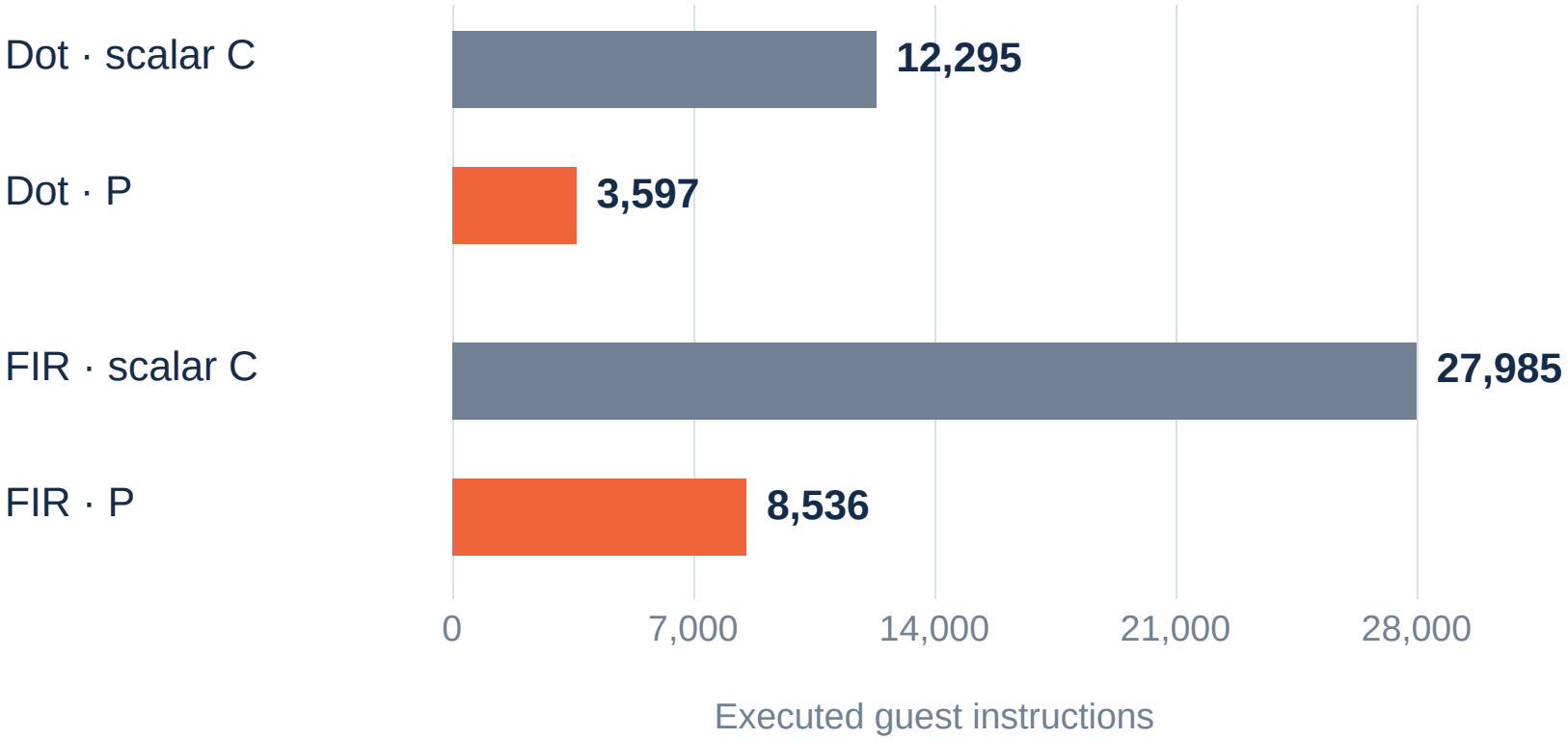
7 asymmetric taps · 64 samples · h[0] multiplies the current sample



Scalar C and P intrinsics produce the same 64 output samples.

Less work for common DSP kernels

RV32 · dot: 1,024 terms · FIR: 32 taps × 64 outputs · base-aligned buffers



RV32 footprint, bytes

Kernel	Text	Frame
DOT C	58	0
DOT P	96	0
FIR C	266	16
FIR P	226	16

Compiler-reported static frames.

RV64 scalar/P reference: dot 7,173 / 1,804; FIR 17,259 / 4,625 instructions.

QEMU guest instructions on prepared inputs, -O3; misaligned word loads assumed for FIR's 2-byte steps. Hardware timing not measured.